

AUTOCHECKLIST: Automated Checklist Refinement for LLM Judges

Mansi Uniyal
Microsoft
Bengaluru, India
mansiuiniyal@microsoft.com

Mukul Singh
Microsoft
Redmond, USA
singhmukul@microsoft.com

Gust Verbruggen
Microsoft
Keerbergen, Belgium
gverbruggen@microsoft.com

Vu Le
Microsoft
Redmond, USA
levu@microsoft.com

Sumit Gulwani
Microsoft
Redmond, USA
sumitg@microsoft.com

Abstract

Evaluating LLM-generated content at scale is a critical challenge in production systems. Structured checklists enable both humans and LLMs to provide detailed, interpretable, and reliable evaluations—but crafting effective checklists remains difficult and often requires iterative refinement based on real-world data. We present AUTOCHECKLIST, a system deployed at Microsoft that augments checklist-based LLM judges with a free-form comment option and dynamically updates the checklist based on this feedback. This ensures checklists adapt to outlier instances and capture dataset-specific criteria that initial designs may miss. We evaluate AUTOCHECKLIST across three production-relevant domains and demonstrate that (1) it consistently surfaces valuable new evaluation criteria, (2) the comment option improves answer quality on the original checklist, (3) it is flexible with respect to seed checklist properties like tone and answer type, and (4) it is robust across different datasets. The results show that automated checklist refinement significantly reduces the manual effort required to build reliable evaluation pipelines.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Machine learning approaches*; • **Software engineering** → Software verification and validation.

Keywords

Large Language Models, Automated Evaluation, AI Agents

ACM Reference Format:

Mansi Uniyal, Mukul Singh, Gust Verbruggen, Vu Le, and Sumit Gulwani. 2026. AUTOCHECKLIST: Automated Checklist Refinement for LLM Judges. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26)*, July 05–09, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3803437.3805263>

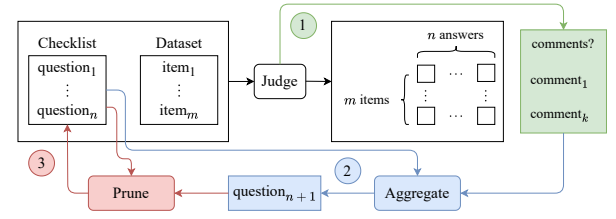


Figure 1: Overall pipeline of AUTOCHECKLIST. Starting with an initialized seed checklist, (1) Generate responses, using the current checklist, and (2) Update the checklist, based on the comments, iteratively.

1 Introduction

Consumption of large language models (LLMs) has increased rapidly in production software systems [3, 12, 20], with LLM-powered agents now deployed across diverse applications. From customer-facing chatbots to internal automation, organizations rely on LLMs for tasks like text summarization [24], machine translation [23], question answering [9], and content generation [16, 21]. Evaluating these outputs at scale remains a critical challenge: automated metrics like BLEU [17] and ROUGE [14] don't capture semantic quality [1], while human judgments are costly, and inconsistent.

LLM-as-a-judge approaches have gained traction as a practical alternative in production environments. They reduce annotation costs, mitigate human biases, and produce evaluations that align closely with human judgments [2]. However, LLM judges still face issues of reproducibility, less transparency in scoring, and bias [7].

A *checklist* can provide more objective and interpretable evaluations by decomposing assessment into specific, verifiable questions [13]. For example, translation checklists can ask about fluency, consistency, and correctness. However, these checklists require manual curation and iteration—a significant burden in practice, as one can never be certain a checklist covers every failure case. What if a translation is ambiguous, biased, or contains jargon?

To address this limitation, we draw inspiration from human surveys, which often include a free-form comment option to capture anything not covered by structured questions. In AUTOCHECKLIST, we augment LLM judges with this comment option, then parse collected comments into new checklist questions, enabling the checklist to evolve with the data. This approach, deployed internally



This work is licensed under a Creative Commons Attribution 4.0 International License. FSE Companion '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN /2026/07
<https://doi.org/10.1145/3803437.3805263>

at Microsoft, significantly reduces the manual effort required to build and maintain evaluation pipelines.

We evaluate AUTOCHECKLIST across three production-relevant domains: translation, summarization, and Excel instruction evaluation. Our results show that AUTOCHECKLIST consistently improve alignment with human judgment. We also analyze sensitivity to seed checklist design and training data composition.

In summary, we make the following contributions:

- (1) A production system for dynamically generating and updating evaluation checklists via free-form comment feedback.
- (2) Extensive analysis demonstrating alignment of generated questions with human judgments and robustness to seed checklist and data variations.
- (3) Qualitative evidence that the comment option improves LLM evaluation accuracy on structured checklists.

2 Related Works

Prior work on automated evaluation spans both task-agnostic testing and LLM-based judging. [19] propose general linguistic capability tests for NLP evaluation that are syntactic and task-agnostic but do not capture subjective or qualitative aspects of natural language generation. CheckEval [13] introduces checklist-based evaluation using LLMs for question augmentation; however, most stages—including aspect definition, question curation, and filtering—remain manual, resulting in static checklists with limited adaptability across domains or data distributions.

Several works study LLMs as judges by optimizing alignment with human scores. [22] grounded numerical ratings using LLM-generated explanations, motivating the use of free-form comments as additional evaluative signals. G-Eval [7], GPTScore [6], and Prometheus 2 [10] aim to maximize correlation or ranking agreement between LLM judgments and human evaluators through static prompts or fine-tuned models. In contrast, our work doesn't seek to optimize evaluator performance but instead measures changes in correlation as a proxy to assess whether new checklist items—derived from model comments—capture meaningful failure modes.

Finally, prior analyses highlight limitations of LLM evaluators. [25] evaluate models as judges under fixed prompts, establishing that stronger models align better with humans, which serves as a foundation for our approach. [4] expose blind spots and inconsistencies in direct scoring, motivating checklist-based evaluation. Our work builds on these insights by proposing an automated, data-driven mechanism to adapt existing checklists over time, rather than relying on static or manually curated evaluation criteria.

3 Background and Motivation

A *checklist* is a list of questions about a type of item, where each question evaluates a boolean or numerical property of the item. Examples of items are translations or summaries of text, and natural language explanations of code. Decomposing the evaluation of these items into concrete questions makes the results more interpretable and objective [15].

EXAMPLE 1. INSTRUCTEXCEL [18] is a dataset of OfficeScript code that performs tasks in Excel and a natural language description for each task, both obtained through crowdsourcing. To ensure quality of the annotations, the ambiguity of each description was scored on

a scale from 0 (invalid) to 3 (valid). Consider the following three descriptions and their scores.

- (d_1) Apply conditional formatting to cell A1 to L112 (3)
- (d_2) Apply conditional formatting > 50000 (2)
- (d_3) Apply conditional formatting from cell C3:C15 of Sheet1 (0)

Despite similar problems, missing either the format or the range, the scores differ greatly. Instead of asking for a single score, we can ask specific questions about properties that indicate ambiguity.

- (q_1) Is the query clear about the desired outcome?
- (q_2) Is the query free from unstated assumptions?
- (q_3) Is the query free from conflicting instructions?

This yields the following interpretable and objective checklist

	q_1	q_2	q_3
d_1	no	no	yes
d_2	no	no	yes
d_3	no	no	yes

Instead of having users answer these questions, they can also be used when an LLM is used as a judge [13]. This allows quick evaluation at scale and thus faster iteration of the item to be evaluated. In the previous example, gpt-4o gives exactly those answers.

Unfortunately, our checklist is not perfect with respect to the descriptions: d_1 and d_3 lack a format and d_3 lacks a range, and there is no question to distinguish between these mistakes. In general, the completeness of a checklist can only be determined based on all suboptimal variations of an item. For example, everything that can be missing from a conditional formatting instruction.

4 AUTOCHECKLIST

We propose to update checklists during evaluation with the simple addition of a free-form *comment* box. An overview of our AUTOCHECKLIST approach is shown in Figure 1. A checklist-based evaluator is shown in black—we can assume that this part is given. In ① green, we show the addition of a comment that asks the judge to provide a comment with anything else that it considers out of the ordinary. In ② blue, we show a component that aggregates the comments into new checklist items. In ③ red, we show an (optional) component that prunes the checklist and removes redundant questions. The next sections describe these components.

Free-form comments. The core idea behind AUTOCHECKLIST is a simple addition to any checklist judging system: asking the judge (an LLM) to add a comment about other properties of a given item that could be interesting for evaluation. In practice, we add the following meta-rules for the comment:

- It is a wrong or correct pattern, a discrepancy, or anything else that stands out to the judge.
- The comment shouldn't explain the response.

EXAMPLE 2. Comments on the InstructExcel dataset:

- **Description:** d_1 , **Comment:** ... condition is missing.
- **Description:** "Highlight the cells which contains $\Sigma\sigma||\epsilon\sigma_i^a$ " in sheet Position and conditions", **Comment:** "the query contains non-standard characters, which adds to the ambiguity."

```
# role
# task
# rules
- Answer with "yes" and "no" for each question.
...
- Comment, if you spot any interesting property for
  evaluation, but is not part of the current checklist.
# checklist
# input
```

Figure 2: An example of **adapting** a judging prompt to output comments with a simple instruction.

Aggregating comments into questions. After collecting comments, we now aggregate them into questions using an LLM. Fittingly, we use a checklist of criteria for the updated checklist.

- Avoid duplicate, redundant, or overlapping questions.
- Each question should only verify one property. For example, “is X clear and concise?” should be converted into two questions “is X clear?” and “is X concise?”
- Avoiding subjective questions.
- Ensure clarity and ease of understanding.

Pruning. We provide the model with the existing checklist and new questions and instruct it to remove questions that are semantically equivalent.

Iterating and stopping. The evaluation and checklist updating procedure can (and should) be repeated for multiple iterations. Our intuition is that the given checklist teaches the model a *concept* of what the item should look like. We stop when there are no more comments or when a maximal number of iterations is reached.

5 Evaluation Setup

5.1 Datasets

Translations. We select the German-to-English NEWS articles, a subset of the WMT22 [11] dataset for machine translation. Each (source, target) pair has a direct assessment [8] annotation on a scale of 0–100 by human judges.

Instructions. The InstructExcel dataset [18] consists of task descriptions that users want to perform in Excel. Each task description was scored on its ambiguity with a score between 0 (completely invalid) and 3 (completely valid) with intermediate scores meaning that ranges or arguments required to perform the task are not given. For example, “*Apply conditional formatting > 50000*” has a score of 2, as it does not specify where to apply it.

Summaries. The SummEval dataset [5] consists of news articles and summaries. It contains scores on aspects (coherence, consistency, fluency, and relevance) obtained from 3 expert and 5 crowd-sourced annotators.

We randomly sample 100 tasks per dataset for our experiments.

5.2 Model and Metrics

We use gpt-4o, set temperature = 0, with a single completion ($n = 1$). Appendix B explores experimentation over other models.

Evaluation Proxy: Change in Correlation. We use weighted correlation to evaluate checklists against human ratings—making the assumption that good checklists can be used to explain these ratings. We can compute a weighted average score $\sum_i^n w_i q_i$ to compute a score from a checklist of n questions. If $\mathbf{w} \in \mathbb{R}^n$ is the vector of n weights, $\mathbf{A} \in \mathbb{N}^{n \times m}$ is the matrix of m answers to all n questions, and $\mathbf{S}$ is the vector of m scores, we minimize $|\mathbf{wA} - \mathbf{S}|$ with respect to $\mathbf{w}$ to find the optimal alignment between the checklist and annotations. We use $\mathbf{w}$ to aggregate checklist answers into scores and compute the Pearson correlation to the human annotations. Note: We use change in correlation as a proxy to estimate whether newly added checklist questions capture meaningful failure modes. Appendix A explores variations of other metrics: Spearman and Kendall metrics.

5.3 Seed checklists

Manual. Manual checklists are extracted from either paper or existing evaluation prompts. For Summaries, we use the checklist of 4 questions on *fluency* from AUTOCHECKLIST [13]. In all other tasks, the manual checklist starts with a single question evaluating the task. For example, in Translations the manual checklist is: “*How is the overall quality of the translated sentence in the context of the source sentence?*”

Synthetic. They are generated by asking the LLM to create a checklist of yes/no questions that evaluates some aspect of a problem domain, for example, the *quality of source text and corresponding translated output*. Without prompting for a number, we see LLM coming up with exactly 10 questions.

```
"Are all key details from source text present in translated text?",
"Is the translated text grammatically correct?",
"Does the translated text maintain same tone and style as source text?",
"Are proper nouns and names correctly translated or transliterated?",
"Is the vocabulary used in the translated text context appropriate?",
"Is the translated text culturally appropriate for the target audience?",
"Is the translated text free from spelling errors?",
"Is the sentence structure in the translated text natural and fluent?",
"Does the translated text accurately convey meaning of the source text?",
"Are idiomatic expressions correctly translated in the target language?"
```

Figure 3: Example seed checklist for Translations.

6 Evaluation

We evaluate how adding free-form comments to a checklist prompt affects the model answers (RQ1), evaluate the effect of different properties of the seed checklist (RQ2) and data subset (RQ3).

6.1 Effect of generating comments

Adding additional checks through free-form comments should not degrade the performance of the checklist. To evaluate this, we generate checklists over all datasets—6 iterations over 5-fold cross-validation sets of 100 data points— with and without comments.

Table 1 shows the partial (% of same outputs) and exact (% of times all answers were the same) matches over cases where the model did and did not provide comments. For example, the partial and exact matches for two lists of answers, [yes, no, yes] and [yes, yes, yes], are 66% and 0% respectively.

We see that AUTOCHECKLIST consistently gets a partial match over 90%, indicating that the model judgement changes only in a

Dataset	Exact (%)		Partial (%)	
	C	℄	C	℄
Translations	61.44	92.33	91.84	98.39
Instructions	70.00	71.61	95.02	95.42
Summaries	60.10	56.42	93.48	94.07

Table 1: Partial (per question) and exact (per checklist) match between checklist answers as a result of adding a comment option, for cases where comments (C) or no comments (℄) were generated.

few cases. As we expect the answers to always match, we analyzed when the model changes its answer and found that in all cases that we encountered, **the answer is better when the model is allowed to make comments**. Our hypothesis is that the comment option does not force the model to cluster everything that can be wrong into one of the checklist questions.

EXAMPLE 3. Consider an Excel *query* “Highlight value where it is greater than 3 on G9:G17 on Sheet1” and a *question* “Does the input query avoid ambiguous language, jargon, or terms that could be interpreted in multiple ways?” Without the comment option, the model answers “no.” This changes to “yes” with the *comment* “the term “highlight” appears to be a typographical error and should likely be “highlight”” which indicates that its negative answer was based on a spelling mistake, not ambiguous language or jargon.

EXAMPLE 4. Consider an Excel *query* “In Calc sheet, Apply color scale from C36 to C 39” and a *question* “Does the input query ensure that cell ranges are specified in a logical order (e.g., from top to bottom, left to right)?” Without comment option, the model answers “no.” This changes to “yes” with the *comment* “the input query has an inconsistent cell reference format (‘C36 to C 39’ should be ‘C36 to C39’)” which indicates that its negative answer was based on cell notation being wrong, not the cell range.

6.2 Effect of seed checklist

We evaluate the impact of each property of the seed checklist: (1) convergence over different seeds, (2) how the expected response type (binary or numerical) affects human alignment, and (3) how the question’s tone affects the tone of the generated questions.

6.2.1 Convergence of checklists. We evaluate the extent to which different seed checklists converge to the same final checklist when evaluating the same dataset.

Metric. To analyze the convergence of the final checklist over various experiments, we manually count # overlapping properties evaluated by checklist questions to compute Jaccard similarity (intersection over union). This enabled handling cases where a question evaluates multiple properties or is worded differently. For example, the below questions taken from 3 different checklists on the Instructions task deliver the same aspect:

- Does the query avoid using vague language such as “some”, “few”, or “many”?
- Does the query avoid using terms that could have multiple meanings in different contexts?

- Does the query avoid using ambiguous terms like “group” without further clarification?
- Is the instruction query free from vague terms that could lead to multiple interpretations?

that are all related to the property of vagueness, and they are thus counted as overlapping.

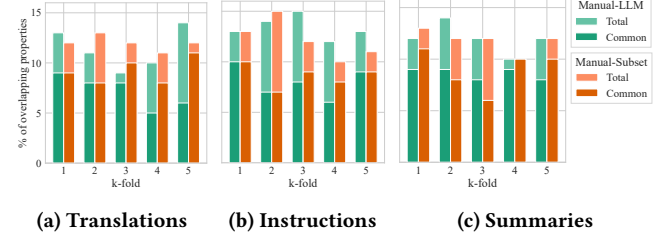


Figure 4: Impact of variation of seed checklist on % overlapping properties of final checklist.

Experiment. We consider three sources of seed checklists: (1) the manual checklist consisting of a single question that evaluates the quality of the output, (2) the synthetically generated checklist using an LLM, and (3) a subset of 50% random questions from the final checklist generated from the manual seed checklist. We perform pairwise comparison between the manual checklist and the other two sources, only comparing the final checklist after at most five iterations, and repeat each experiment for five folds on different data splits (80/20 cross validation, where the test set is ignored in this experiment).

Results. Figure 4 illustrates the % of overlapping properties. Starting from a single question, the manual seed gives shortened checklists on average, yet covers most of the properties covered when starting with a larger checklist. Some of these general questions that are not generated via the manual seed checklist are “Does the instruction query provide a clear context or background information if needed?” and “Is the desired outcome or result clearly stated in the instruction query?”

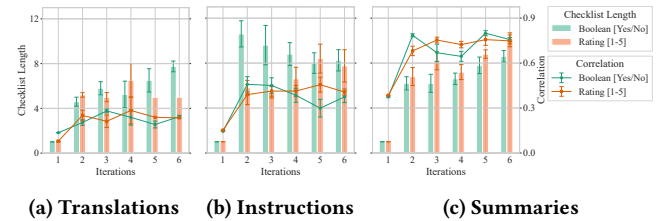


Figure 5: Impact of response type

6.2.2 Response Format. We explore the impact of response formats on the quality of the checklists. We consider two popular itemized evaluation formats: (1) checklists (boolean with yes and no) and (2) scorecards (1-5 rating).

Figure 5 shows the length and correlation with human ratings for the checklists over increasing iterations for different response

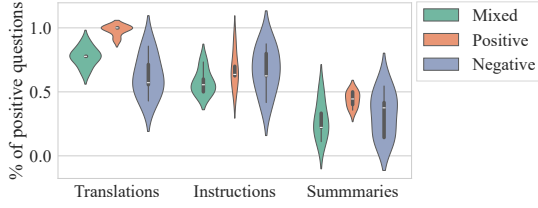


Figure 6: Impact of tone of the seed checklist on the tone of the final checklist

types. We see that the correlation is comparable for both response formats, showing the stability of AUTOCHECKLIST. Furthermore, upon qualitative analysis of the checklists, we find that the checks of both response types converge to be very similar.

6.2.3 Tone. LLMs have inherent biases that affect the generations based on the tone of questions asked. For example, consider a positively phrased question “*is the summary concise?*” versus the negatively phrased version of this question “*does the summary avoid being lengthy?*”. Though both these questions will have the same answer for a good summary, for ambiguous cases, the phrasing affects the model’s ability to assess these. To evaluate the impact of the tone of the checklist, we manually create three versions of checklists with varying tones in the seed checklist: (1) positive, (2) negative, and (3) mixed (original).

Figure 6 shows the impact of the tone of the seed checklist on the tone of the final checklist. We observe that the final checklist is consistently mixed tone, regardless of the seed checklist tone. Starting with a positive-toned seed checklist does provide an edge in generating higher % of positive-toned final checklists than other variations. Furthermore, tasks like Translations having a higher % of positive questions show how AUTOCHECKLIST adapts its tone based on the task.

6.3 Effect of training dataset

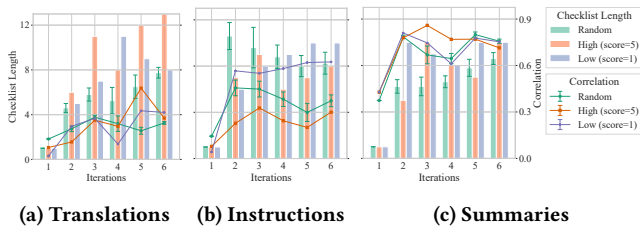


Figure 7: Impact of training data

We analyze the impact of training dataset composition on the quality of the final checklist. For this, we consider three training scenarios: (1) using a 5-fold cross-validation with randomly selected samples, (2) training on the 80 best-scored samples based on human annotations, and (3) training on the 80 worst-scored samples. In each case, the checklist is updated based on the comments generated in response to the training samples. Figure 7 summarizes the checklist length (bars) and correlation with human annotations (lines) for increasing iterations for the different datasets.

We find notable differences in convergence behavior. The results show that random sampling achieves the best convergence to the human annotations across the evaluation metrics. In contrast, training on low-scoring samples leads to a larger checklist, with many questions being overly specific or uncommon, and a lower overall convergence to the human annotations. Thus, the model struggles to generalize effectively from weaker samples. On the other hand, training on high-scoring samples results in a checklist that converges similarly to the random sampling case but with more fluctuation in the metrics across iterations. This instability makes it difficult to optimize the aggregation of individual responses into a coherent score, as the model lacks balanced training data—being trained solely on either very high or very low-scoring examples limits its ability to generalize to new test sets.

6.4 Effect of pruning

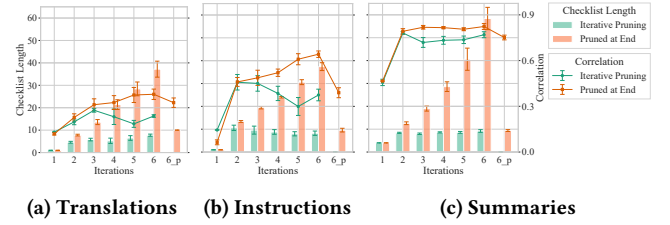


Figure 8: Impact of pruning

We find a key trade-off between iterative pruning and pruning at the end: iterative pruning leads to a more concise checklist with controlled convergence, while pruning only once after full expansion results in broader coverage but some loss in quality. Iterative pruning ensures a gradual convergence of scores, consistently keeping the checklist within a range of around 10 questions, balancing conciseness with effectiveness. Figure 8 shows the checklist length (bars) and correlation with human annotations (line) on increasing iterations for iterative and prune-at-end strategies.

In contrast, when the checklist is allowed to expand fully over multiple iterations before being pruned, it covers a wide array of potential questions and converges quickly toward the GT score. However, after pruning this comprehensive version, the checklist is reduced to a more summarized form, which sacrifices quality. Interestingly, both approaches lead to checklists that share 7/8 core properties, demonstrating that while the number of questions and phrasing differ, both methods converge to capture similar aspects.

7 Conclusion

We propose AUTOCHECKLIST, to automate the checklist generation while focusing on converting subjective aspects to more like sub-objective questions, providing more interpretable, robust, and reproducible results. It leverages potential errors in the dataset and dynamically updates via comments as feedback, making it tailored to the dataset application. Using the pipeline, we are not only able to generate scores quite aligned with the human ones but also showcase its applicability on various tasks and detailed analysis on robustness to variation in different components.

References

- [1] Chris Callison-Burch, Miles Osborne, and Philipp Koehn. 2006. Re-evaluating the Role of Bleu in Machine Translation Research. In *11th Conference of the European Chapter of the Association for Computational Linguistics*, Diana McCarthy and Shuly Wintner (Eds.). Association for Computational Linguistics, Trento, Italy, 249–256. <https://aclanthology.org/E06-1032>
- [2] Cheng-Han Chiang and Hung-yi Lee. 2023. Can Large Language Models Be an Alternative to Human Evaluations?. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 15607–15631. doi:10.18653/v1/2023.acl-long.870
- [3] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jia Shi, Jiawei Wang, Jingcheng Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiusi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shutong Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhen Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- [4] Sumanth Doddapaneni, Mohammed Safi Ur Rahman Khan, Sshubam Verma, and Mitesh M Khapra. 2024. Finding blind spots in evaluator llms with interpretable checklists. arXiv preprint arXiv:2406.13439 (2024).
- [5] Alexander R Fabbri, Wojciech Kryściński, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir Radev. 2021. Summeval: Re-evaluating summarization evaluation. *Transactions of the Association for Computational Linguistics* 9 (2021), 391–409.
- [6] Jinlan Fu, See Kiong Ng, Zhengbao Jiang, and Pengfei Liu. 2024. Gptscore: Evaluate as you desire. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6556–6576.
- [7] Mingqi Gao, Xinyu Hu, Jie Ruan, Xiao Pu, and Xiaojun Wan. 2024. LLM-based NLG Evaluation: Current Status and Challenges. arXiv:2402.01383 [cs.CL] <https://arxiv.org/abs/2402.01383>
- [8] Yvette Graham, Timothy Baldwin, Alistair Moffat, and Justin Zobel. 2013. Continuous measurement scales in human evaluation of machine translation. In *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*. 33–41.
- [9] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering*. 1219–1231.
- [10] Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024. Prometheus 2: An open source language model specialized in evaluating other language models. arXiv preprint arXiv:2405.01535 (2024).
- [11] Tom Kocmi, Rachel Bawden, Ondřej Bojar, Anton Dvorkovich, Christian Federmann, Mark Fishel, Thamme Gowda, Yvette Graham, Roman Grundkiewicz, Barry Haddow, et al. 2022. Findings of the 2022 conference on machine translation (WMT22). In *Proceedings of the Seventh Conference on Machine Translation (WMT)*. 1–45.
- [12] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.
- [13] Yukyung Lee, Joonghoon Kim, Jaehye Kim, Hyowon Cho, and Pilsung Kang. 2024. CheckEval: Robust Evaluation Framework using Large Language Model via Checklist. arXiv preprint arXiv:2403.18771 (2024).
- [14] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*. Association for Computational Linguistics, Barcelona, Spain, 74–81. <https://aclanthology.org/W04-1013>
- [15] Yixin Liu, Alex Fabbri, Pengfei Liu, Yilun Zhao, Linyong Nan, Ruilin Han, Simeng Han, Shafiq Joty, Chien-Sheng Wu, Caiming Xiong, and Dragomir Radev. 2023. Revisiting the Gold Standard: Grounding Summarization Evaluation with Robust Human Evaluation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 4140–4170.
- [16] OpenAI. 2023. Chat-GPT. <https://openai.com/blog/chatgpt> last-accessed: Oct 2023.
- [17] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Pierre Isabelle, Eugene Charniak, and Dekang Lin (Eds.). Association for Computational Linguistics, Philadelphia, Pennsylvania, USA, 311–318. doi:10.3115/1073083.1073135
- [18] Justin Payan, Swaroop Mishra, Mukul Singh, Carina Negreanu, Christian Poeltz, Chitta Baral, Subhro Roy, Rasika Chakravarthy, Benjamin Van Durme, and Elnaz Nouri. 2023. InstructExcel: A Benchmark for Natural Language Instruction in Excel. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 4026–4043.
- [19] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. arXiv preprint arXiv:2005.04118 (2020).
- [20] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code llama: Open foundation models for code.
- [21] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1*. 27–43.
- [22] Many Wadhwa, Jifan Chen, Junyi Jessy Li, and Greg Durrett. 2023. Using natural language explanations to rescale human judgments. arXiv preprint arXiv:2305.14770 (2023).
- [23] Biao Zhang, Barry Haddow, and Alexandra Birch. 2023. Prompting large language model for machine translation: A case study. In *International Conference on Machine Learning*. PMLR, 41092–41110.
- [24] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. 2024. Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics* 12 (2024), 39–57.
- [25] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

iter.	Pearson's Corr.	Spearson's ρ	Kendall's τ
1	0.353	0.354	0.659
2	0.771	0.640	0.790
3	0.710	0.645	0.793
4	0.838	0.734	0.843
5	0.839	0.798	0.872
6	0.814	0.785	0.861

Table 2: Pruned at end

iter.	Pearson's Corr.	Spearson's ρ	Kendall's τ
1	0.708	0.540	0.740
2	0.648	0.587	0.757
3	0.670	0.511	0.727
4	0.478	0.460	0.701
5	0.610	0.590	0.767
6	0.543	0.507	0.730

Table 3: Pruned iteratively

A Additional metrics

While using Pearson correlation is acceptable, incorporating ranking-based correlation metrics provides a more comprehensive analysis.

Note that we are not trying to optimize the correlation: we are using the change in correlation as a proxy metric to evaluate the relevance of newly added questions (which we will add more explicitly, as that might not have been entirely clear). We have computed these other (Spearman and Kendall) metrics, and their relative results are consistent. If the reviewers have a strong preference towards either of these metrics, we are happy to replace it—but adding multiple metrics does not change our findings. Table 2 and 3 displays the comparisons of change in metrics for the Instructions dataset.

B LLM variation

We have performed qualitative experiments on other models (gpt-35-turbo and gpt-4-turbo) and did not find their performance satisfactory—even in the base evaluation task. We recognize this limitation and will explicitly add it. Specifically, some of the challenges we face with smaller models are

- The responses do not follow the expected structure, which makes it hard to parse in an automated way, and hallucinations occur where the model does not even provide answers but simply repeats the task description.
- Using smaller models for evaluation is viable through fine-tuning (like Prometheus 2), but this would likely have a negative effect on the ability to evaluate out-of-distribution mistakes—exactly the ones that we aim to catch with the free-form comment option.

C Limitations

Whereas we find the added questions to be valuable and correlated to human ratings, we suggest that users remain in the loop to verify them. If the dataset is large, the evaluation may potentially not be repeated for multiple iterations. LLMs remain prone to hallucinations, and even checklists are not infallible, but the right checklist is a step forward to direct assessment.

D Future Works

Future work includes incorporating lightweight human-in-the-loop validation to further improve the quality of generated checklist items and extending AutoChecklist to multi-modal and structured domains such as code and tables. Exploring adaptive weighting of checklist criteria during refinement may further enhance alignment with human judgments. Additionally, evaluating the approach in long-term, real-world deployments will help better understand its scalability and robustness across evolving datasets.